
I-NutriGuide Application Analysis Handoff

Architecture, data, recommendation, admin, and mobile handoff

Generated: May 30, 2026

Audience: another ChatGPT or reviewer analyzing the current I-NutriGuide application without first reading the entire repository.

Scope: local development build, Docker services, Django backend, React admin panel, Expo mobile app, food image metadata, supplement normalization, association rules, safety constraints, and recent UI/runtime fixes.

Purpose

This PDF is a compact handoff for another ChatGPT session to analyze the current I-NutriGuide application. It summarizes the architecture, runtime services, data imports, recommendation engine, admin panel, mobile app, and recent changes.

Sensitive local credentials are intentionally omitted. Use the repository and environment files on the local machine for private operational details.

Project Shape

- Monorepo root: c:/Users/hp/Desktop/inutriguide_codex_docs
- Backend: apps/backend, Django 5.1 + Django REST Framework.
- Admin panel: apps/admin-panel, React + Vite.
- Mobile app: apps/mobile-app, Expo Router + React Native Web.
- Shared packages: packages/shared-types and packages/config.
- Infrastructure: Docker Compose with PostgreSQL, Redis, Neo4j, backend, admin panel, mobile app, Celery worker/beat, and Dozzle.

Local Runtime

- Backend API: http://localhost:8000
- Backend health: http://localhost:8000/api/v1/health/
- Admin panel: http://localhost:5173
- Mobile web / Expo: http://localhost:8081
- PostgreSQL: localhost:5432
- Redis: localhost:6379
- Neo4j browser: http://localhost:7474

- Main startup command used during development: `docker compose -f docker-compose.dev.yml up -d --build backend admin_panel mobile_app`

Current Database Snapshot

- Foods: 3,263 active of 3,263 total.
- Supplements: 106 catalog rows.
- Nutrients: 84.
- Supplement normalizations: 105.
- Canonical supplement categories: 24.
- Food-supplement seed synergy rules: 161.
- Safety constraints: 8.
- Association transactions: 1,200 with 9,029 transaction items.
- Mined association rules: 80.
- Synced scoring rules: 170.
- Nutrient interactions: 10.
- Recommendation runs: 29 with 191 recommendation items.
- Saved recommendation foods: 1.

Backend Domain Model

- Food catalog models include `FoodCategory`, `Food`, and `FoodNutrient`.
- Food now stores `image_path`, `image_alt`, `recommended_for_supplements`, `nutrient_tags`, `synergy_reason`, `avoid_or_caution`, `allergen_tags`, `diet_tags`, `association_rule_items`, and `is_active`.
- Recommendation history is stored in `RecommendationRun`, `RecommendationItem`, and `SavedRecommendationItem`.
- Association-rule dataset models include `SupplementCategory`, `SupplementNormalization`, `FoodSupplementSynergyRule`, `SafetyConstraint`, `AssociationTransaction`, `AssociationTransactionItem`, and `MinedAssociationRule`.
- Legacy/synced scoring rules are stored in `AssociationRule`.
- Safety constraints are stored separately from positive synergy rules and should not become positive recommendations.

Recommendation Logic

- Do not replace the current recommendation engine. The active logic is a hybrid recommender.
- Core ranking blends content-based filtering, nutrient/supplement matching, association-rule scoring, and collaborative preference signals from recommendation/feedback history.
- User supplements are normalized before rule matching using `SupplementNormalization` and `SupplementCategory` canonical items such as `supp:iron` or `supp:vitamin_c`.
- Positive association seed rules and mined rules can raise `rule_score` and add explanations.
- Medical/allergy/safety filters must remain stronger than positive association scores.
- Responses include food image metadata, score breakdown, confidence, matched supplement/rule metadata, synergy explanation, warnings, and feedback actions.

Data Import Commands

- Food images CSV command: `python manage.py import_food_images_seed data/i_nutriguide_food_images_seed.csv`
- Association Excel import: `python manage.py import_association_dataset "data/I-NutriGuide_association_rule_dataset that i creat.xlsx"`
- Mining command: `python manage.py mine_association_rules`
- Rule refresh command: `python manage.py refresh_recommendation_rules`
- Recommended post-import flow: `makemigrations`, `migrate`, `import dataset`, `mine rules`, `refresh rules`, `run tests`.

API Surface To Inspect

- Authentication: `/api/v1/auth/login/`, `/api/v1/auth/me/`, `/api/v1/auth/refresh/`.
- Recommendations: `/api/v1/recommendations/foods/?n=10`, `/api/v1/recommendations/generate/`, `/api/v1/recommendations/preview/`.
- Admin dashboard: `/api/v1/admin/dashboard/`.
- Admin recommendation review: `/api/v1/admin/recommendations/`.
- Rule dataset admin APIs: association supplement categories, supplement normalizations, synergy seed rules, safety constraints, mined association rules, and association transactions.
- Knowledge APIs: foods, nutrients, supplements, nutrition interactions.

Admin Panel

- Routes include Dashboard, Foods, Add Food, Knowledge Base, Rules, Rule Dataset, Recommendations, Feedback, Chats, and Users.
- Dashboard now emphasizes Supplement Knowledge, normalization map, seed rules, mined rules, safety constraints, transactions, interactions, recommended foods, and saved foods.
- Knowledge Base supplements page now shows the supplement catalog plus imported normalization aliases and canonical supplement categories.
- Knowledge Base interactions page now combines nutrient interactions and imported supplement safety constraints.
- Food admin supports image preview and image metadata fields.
- The admin frontend API base is local: `http://localhost:8000/api/v1`.

Mobile App

- Expo Router tab bar includes Home, Recs, Supps, Chat, and Profile.
- Supps tab uses a medkit icon and opens `/tabs/supplements`.
- Add supplement flow remains hidden at `/tabs/supplements-new`.
- Profile screen includes a bottom Log out button that clears session storage and routes to the welcome screen.
- Recommendation cards display food images, food name, match percentage, synergy reason, supplement connection, and safety note when available.

- Relative media paths from the backend should be prefixed with the backend media URL, with fallback image support.

Brand And UI Direction

- App identity: clean wellness/nutrition style, green primary color, orange accents, rounded translucent cards, soft shadows, healthy food photography.
- Auth screens should reuse the welcome/get-started healthy-food background image with resizeMode cover and a soft overlay.
- Avoid creating disconnected designs for onboarding/auth/admin/mobile surfaces.
- Mobile pages should avoid overflow on small screens and keep forms readable.

Recent Fixes

- Removed stale API URL pointing to 10.245.156.116 and rebuilt local services.
- Started backend, admin panel, and mobile app on localhost ports 8000, 5173, and 8081.
- Updated admin dashboard and knowledge-base UI for imported association data.
- Added the mobile Supps bottom-tab medkit icon.
- Added a bottom Log out button on the mobile Profile screen.

Suggested Analysis Prompt

Use this prompt with another ChatGPT:

Analyze the I-NutriGuide application from this handoff. Focus on architecture quality, recommendation correctness, safety handling, data model consistency, admin usability, mobile UX, API completeness, test gaps, deployment risks, and next best improvements. Do not propose replacing the hybrid recommender; suggest incremental fixes that fit the current Django/React/Expo stack.

High-Value Review Questions

- Are supplement normalizations used consistently in all recommendation entry points?
- Do safety constraints and allergy checks always override positive association rules?
- Are image_path and fallback media handling consistent across backend, admin, and mobile?
- Are admin counts and page labels clear enough for non-technical data management?
- Are mobile navigation, logout, supplement routines, and recommendation details ergonomic on small screens?
- Which tests should be added first for the highest risk paths?
- What production security settings need review before deployment?